DETAILED PROBLEM STATEMENT

Comparison of Two General Trees

1. Introduction

Trees are non-linear data structures used to represent hierarchical relationships. A general tree allows each node to have any number of children. Comparing two general trees is useful when checking whether two hierarchical representations contain the same arrangement of nodes and the same values.

2. Problem Definition

Write a C++ program to construct two general trees and determine whether they have the same structure and corresponding node values. Each node stores a value and a list of child pointers. The user supplies the node values and the parent-to-children relationships for each tree. The program compares the roots first, then recursively compares corresponding child nodes in order. The trees are considered the same only when their corresponding node values match and each pair of corresponding nodes has the same number of children arranged in the same order.

3. Need for the System

Tree comparison is a common operation in hierarchical data processing. It can be used to compare organization charts, category hierarchies, expression trees, and other tree-based representations. This project demonstrates recursive traversal and structural comparison of general trees.

4. Objectives

- Construct two general trees from user input.
- Represent each node with a value and a list of children.
- Check whether both trees have the same number and arrangement of children.
- Compare corresponding node values.
- Use recursion to compare subtrees.
- Display whether the trees are identical in structure and values.

5. Proposed Working

The program first reads the nodes and child relationships for Tree 1, then repeats the process for Tree 2. Node 1 is treated as the root. The program validates that every other node has exactly one parent, that all nodes are reachable from the root, and that the relationships contain no cycles. It then compares the roots. If their values or child counts differ, the trees are not identical. Otherwise, each pair of corresponding children is compared recursively. If every comparison succeeds, the trees are reported as identical.

6. Functional Requirements

- Read node values for both trees.
- Read the number of children and child IDs for each node.
- Validate tree relationships and reject invalid input structures.
- Compare node values and child counts recursively.
- Print a clear result indicating whether the trees are the same.

7. Scope and Assumptions

This program treats a general tree as an ordered tree: the order in which children are entered matters. For example, a node with children A then B is considered different from a node with children B then A. The comparison checks both structure and values; it does not only compare the number of nodes. Values are read as single tokens without spaces.

8. Expected Outcome

The program should correctly identify two trees as identical when all corresponding values and child arrangements match, and identify them as different when a value, child count, or child arrangement differs.